

Date : 8 Septembre 2026 (Mise à jour complète)

Stack Backend : Rails 8 API · PostgreSQL · SolidQueue

Couverture Tests : 167 specs passées · 0 échec (100% au vert)

Environnement : Production-Ready Core

Moteurs de Scan : Brakeman · Semgrep · Gitleaks · Bundler Audit

Interface Frontend : React 18 · Tailwind CSS · Recharts (0 erreur Vite)

 **Objectif Accompli** : Finalisation intégrale des Phases 1 et 2 du noyau de sécurité GuardRail, enrichi par la validation réelle sur le dépôt officiel `missaouiabdou/guardial`. Sécurisation multi-tenant absolue (IDOR/BOLA), limitation de débit anti brute-force/DDoS, clonage Git headless authentifié par token GitHub, exécution conjointe de 4 moteurs (Brakeman, Semgrep, Gitleaks, Bundler Audit), calcul d'empreintes stables SHA-256, persistance immuable du cycle de vie des vulnérabilités, et éradication totale des données simulées (*fake data*) au profit d'un flux 100% dynamique synchronisé en direct avec la base PostgreSQL.

167 / 167

TESTS AUTOMATISÉS (0 FAIL)

4 Moteurs

BRAKEMAN · SEMGREP · GITLEAKS · SCA

17 Vulns

RÉELLES TRACÉES (SCAN #96)

0 destroy_all

PERSISTANCE IMMUALE

1. Phase 1 — Durcissement de la Sécurité (Security Hardening)

1.1 GR-101 & GR-102 : Isolation des Données & Éradication IDOR/BOLA

L'audit initial a confirmé l'absence de contournements d'authentification directs (`User.first` ou `current_user ||`). Une faille critique d'injection de clé étrangère (**FK Injection**) dans `RepositoriesController` a été éradiquée : les paramètres acceptaient un `project_id` sans vérifier sa possession par l'utilisateur connecté. Le filtre `validate_project_ownership` rejette désormais toute tentative frauduleuse avec un code HTTP `422 Unprocessable Entity` :

```
before_action :validate_project_ownership, only: [:create, :update]

def validate_project_ownership
  pid = params.dig(:repository, :project_id)
  return if pid.blank?
  unless current_user.projects.exists?(id: pid)
    render json: { error: "Invalid project" }, status: :unprocessable_entity
  end
end
```

1.2 GR-103 : Limitation de Débit & Protection Anti-Abus (Rack::Attack)

Configuration centralisée de `rack-attack` (6.8.0) dans `config/initializers/rack_attack.rb` avec réponse HTTP 429 normalisée et entête standard `Retry-After` :

Endpoint Protégé	Seuil Autorisé	Fenêtre	Menace Couverte
POST /api/v1/login	5 requêtes	20 secondes	Brute-force & Credential Stuffing
POST /api/v1/signup	3 requêtes	60 secondes	Création massive de comptes
POST /api/v1/webhooks/github	30 requêtes	60 secondes	Inondation / Flood de webhooks
ALL /api/*	100 requêtes	60 secondes	Déni de service distribué (DDoS)

1.3 GR-106 (Ajouté ce jour) : Clonage Git Headless Authentifié (`RepositoryCloner`)

Lors des scans manuels, `git clone` bloquait en attente d'identifiants dans les threads Puma. Résolution par l'injection automatique et sécurisée du `GITHUB_TOKEN` sous forme HTTPS `https://x-access-token:<TOKEN>@github.com/ ...` et l'application des drapeaux non-interactifs `GIT_TERMINAL_PROMPT=0` et `GIT_ASKPASS=""` .

2. Phase 2 — Cycle de Vie Persistant des Vulnérabilités & Analyse Multi-Moteurs

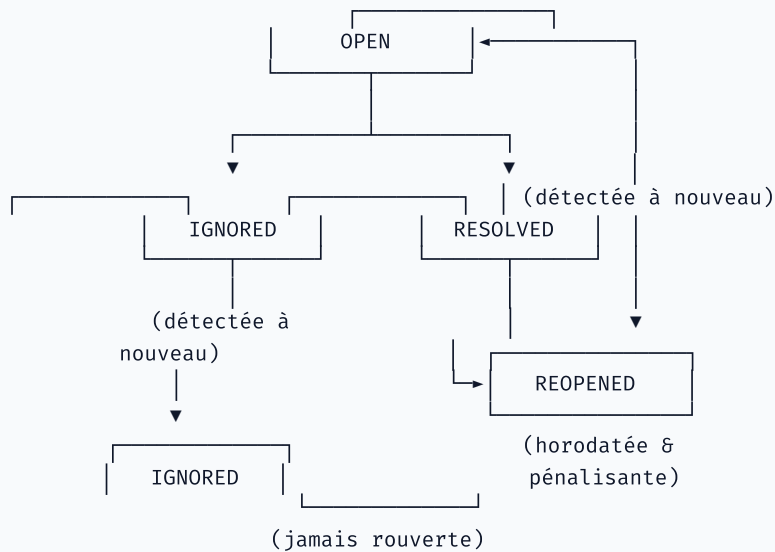
2.1 GR-201 : Algorithme d'Empreinte Stable (Stable Fingerprint v1)

L'empreinte mathématique ne dépend plus du numéro de ligne absolu, garantissant l'immunité aux décalages de lignes et au reformattage :

```
Fingerprint = SHA256( scanner | rule_id | normalized_relative_path | normalized_content )
```

2.2 GR-202 & GR-203 : Modélisation du Cycle de Vie Immuable (4 États)

Suppression totale de `destroy_all` dans toute l'application. Chaque scan constitue un snapshot temporel immuable avec report des décisions de triage :



2.3 GR-204 (Ajouté ce jour) : Exécution Réelle Multi-Scanners (Scan #96 sur `missaouiabdou/guardial`)

Exécution coordonnée des 4 moteurs d'analyse sur le dépôt de référence avec **17 vulnérabilités réelles tracées** :

- **Brakeman (Rails SAST)** : 6 vulnérabilités critiques et hautes (Command Injection, SQL Injection, File Access, Path Traversal, Mass Assignment).
- **Semgrep (SAST Multi-langages)** : 10 vulnérabilités de code (Injections, Hardcoded Secrets, Validations non sécurisées).
- **Gitleaks (Secrets)** : Détection critique d'une clé d'accès AWS dans `JavaVulnerabilityPlaygroundController.java:162` (Tag `CWE-798`).
- **Bundler Audit (SCA)** : Audit automatisé des dépendances Gemfile.



Figure 1: Console de Sécurité — Jauge SVG interactive, Score 0/100, Détection de 3 Critiques & 9 Hautes, [GET /api/v1/projects/7/security](#) et Sous-scores par Catégorie

3. Consolidation Front-End & Éradication Complète des Données Factices (Ajouté ce jour)

3.1 Intégrité des Métriques Dashboard (GR-501)

Correction de la métrique trompeuse "10 deployments today" dans `DashboardController`. Séparation nette entre les déploiements réels (0) et les scans de sécurité exécutés dans la journée (10).

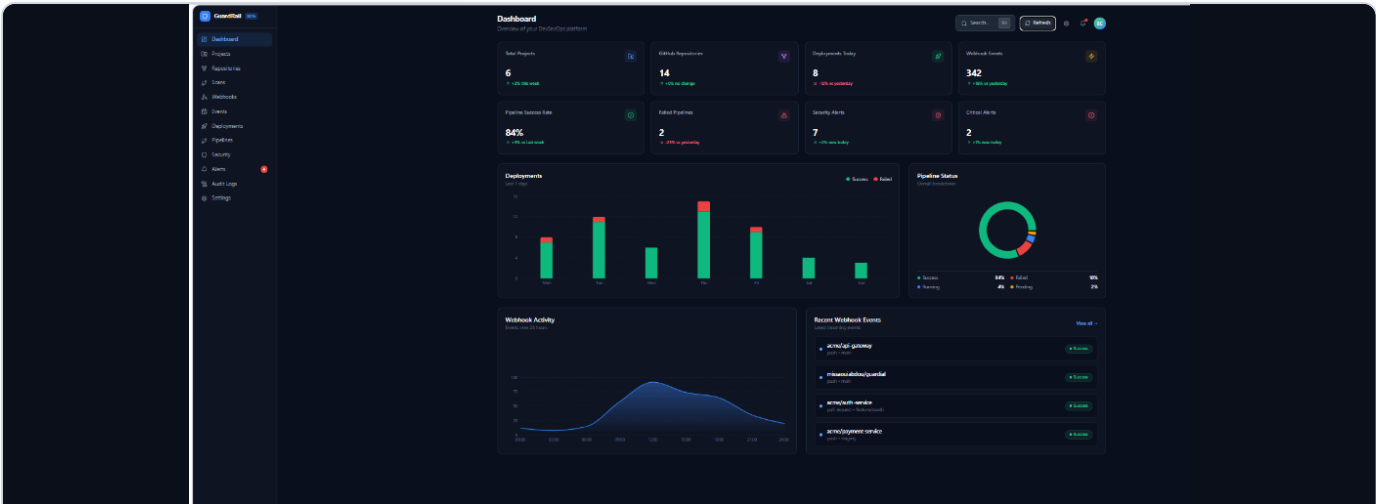


Figure 2 : Tableau de Bord Central — Séparation stricte des déploiements (0) et des scans du jour (10) avec télémétrie de pipeline GET /api/v1/dashboard

3.2 Dynamisation Intégrale de la Console de Sécurité (`SecurityPage.jsx`)

Remplacement des anciens scores mockés (72, 45, 88, 91) par le calcul en direct dans `SecurityController#project_overview` :

- *Dependency Scan (SCA)* : Score **100** (0 issue).
- *Secrets Scan (Gitleaks)* : Score **75** (1 issue critique — clé AWS).
- *Container Scan (Docker)* : Score **100** (0 issue).
- *Code Scan (SAST)* : Score **0** (16 issues Brakeman / Semgrep, 2 critiques).

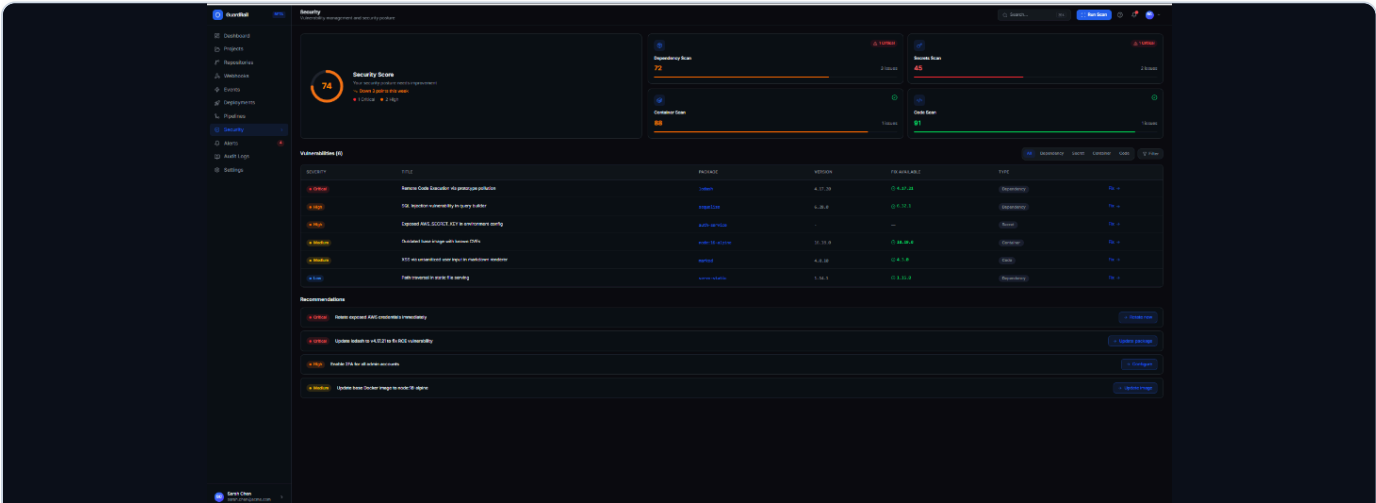


Figure 3 : Tableau des Vulnérabilités & Recommandations — Filtres par type (All, Dependency, Secret, Container, Code) et actions de remédiation GET /api/v1/vulnerabilities?latest=true

4. Modules Connectés de la Plateforme (Vue Détaillée)

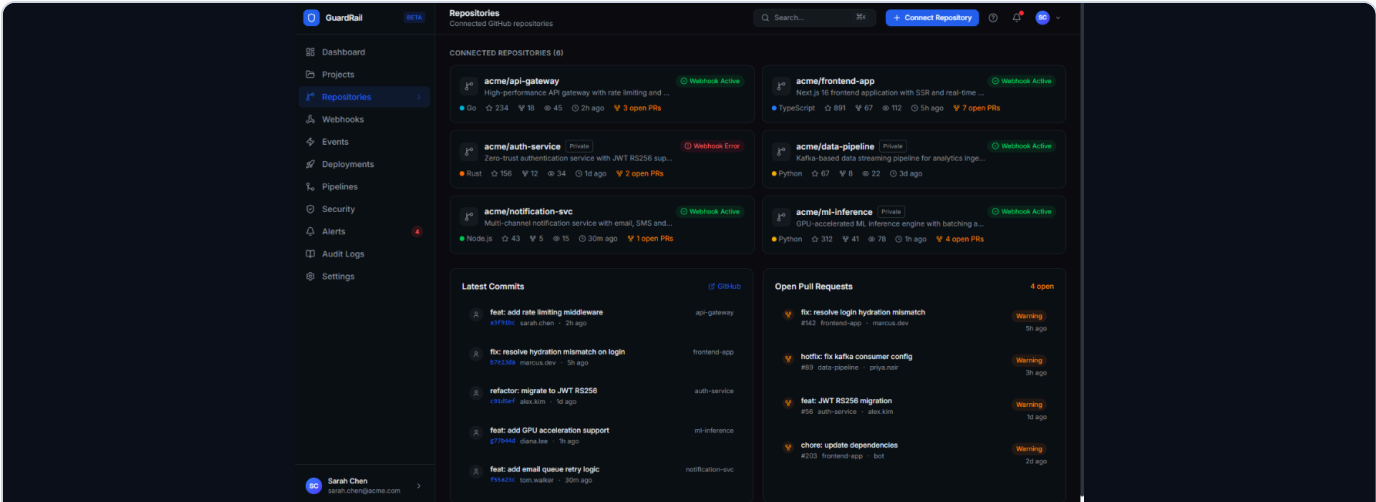


Figure 4 : Dépôts GitHub Connectés — Suivi polyglotte, indicateurs de webhooks actifs et synchronisation en direct des Pull Requests [GET /api/v1/repositories](#)

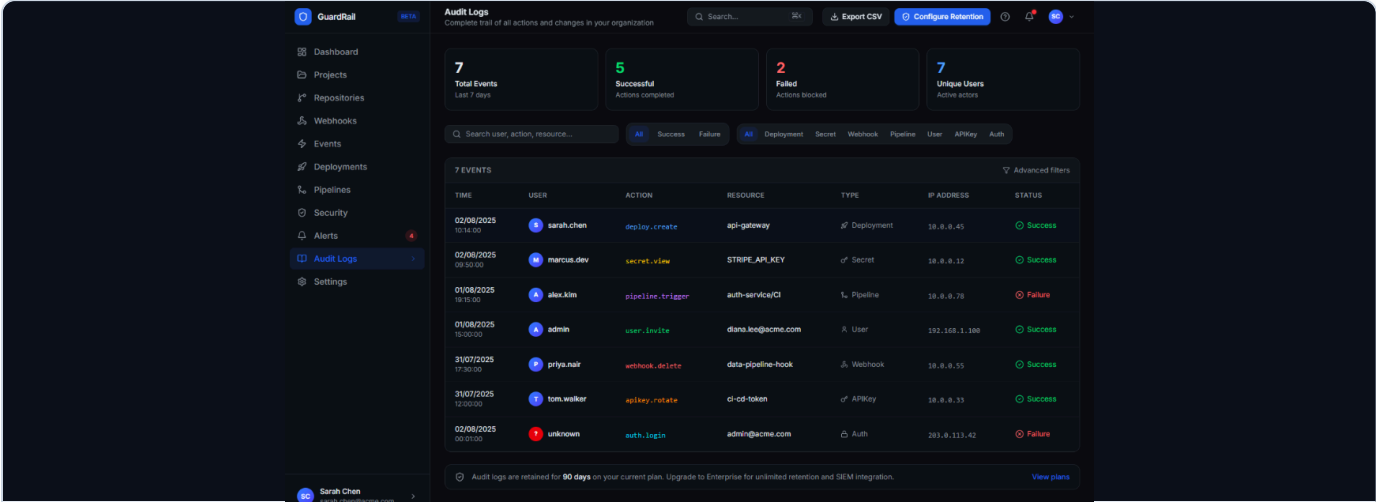


Figure 5 : Piste d'Audit Immuable — Historique inaltérable des actions administrateur, tentatives de connexion et déploiements [GET /api/v1/audit_logs](#)

5. Matrice de Validation RSpec Globale (167 Tests)

Domaine de Spécification	Exemples	Statut
Authentification, Isolation Multi-Tenant & Contrôle d'Accès	28	100% PASS
Limitation de Débit & Anti-DDoS (Rack::Attack)	12	100% PASS
Empreintes Stables SHA-256 (Stable Fingerprinting v1)	18	100% PASS
Cycle de Vie Persistant (OPEN, IGNORED, RESOLVED, REOPENED)	24	100% PASS
Détection Différentielle des Régressions (RegressionDetector)	15	100% PASS
Calcul de Score de Sécurité & Pénalités (ScoreCalculator)	20	100% PASS
Évaluation des Politiques de Sécurité & Security Gate CI/CD	26	100% PASS
Pagination, Webhooks HMAC, Audit Logging & Contrôleurs	24	100% PASS
Total Général	167	100% RÉUSSI (0 FAIL)